GOBIERNO DEL
ESTADO DE MÉXICO

UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO

# UNIVERSIDAD MEXIQUENSE DEL BICENTENARIO

## UNIDAD DE ESTUDIOS SUPERIORES

SAN JOSÉ DEL RINCÓN

Periodo: 25-26/1

## EXAMEN PRIMER PARCIAL "TRASLACIÓN DE FIGURAS"

CARRERA:

**INGENIERÍA EN SISTEMAS COMPUTACIONALES**

ASIGNATURA: **GRAFICACIÓN**

PRESENTA:

**MAURICIO NOLAZCO LONJINO**

**CRISTOFER PIÑA RODRÍGUEZ**

SEMESTRE: **SÉPTIMO**

GRUPO: **13SC171**

DOCENTE:

**I.I. ANAYELI TAPIA TÉLLEZ**

***24o. SEPTIEMBRE DE 2025.***

# Índice

Introducción ..... 1

Desarrollo ..... 2

Herramientas a usar: ..... 2

Pruebas ..... 12

Conclusiones ..... 17

# Índice de figuras

Figura 1. Archivos html,css,js ..... 2

Figura 2. Primera parte del código del html. ..... 3

Figura 3. Segundo bloque de código de html. ..... 4

Figura 4. Tercer bloque del código html. ..... 4

Figura 5. Primer bloque de código de JS. ..... 5

Figura 6. Segundo bloque de código de JS. ..... 5

Figura 7. Tercer bloque de código de JS. ..... 6

Figura 8. Cuarto bloque de código de JS ..... 7

Figura 9. Quinto bloque de código de JS. ..... 8

Figura 9. Sexto bloque de código de JS. ..... 9

Figura 10. bloque de código de CSS. ..... 10

Figura 11. Ejecución de la página web. ..... 12

Figura 12. Ejecución de la página web. ..... 12

Figura 13. Muestra de los puntos para la graficación en la página web. ..... 13

Figura 14. Muestra de los puntos en el plano cartesiano para la graficación en la página web. ..... 14

Figura 15. Ejemplo de una traslación de una figura en la página web. ..... 14

Figura 16. Ejemplo traslación de una figura en la página web. ..... 15

Figura 17. Ejemplo traslación de una figura en la página web. ..... 15

# Introducción

Como parte de la evaluación del primer parcial en la asignatura de Graficación el docente pidió como parte del examen hacer un proyecto sobre la traslación de figuras en equipos, usando la herramienta que mas se nos adecua, cabe mencionar que una vez elegidas las herramientas ya no se podía cambiar y se quedaba como esa opción.

Como parte de la actividad lo que se tenia que hacer como en general una traslación de una figura que se pueda graficar.

# Desarrollo

Como parte inicial dependiendo de la herramienta que se use lo que debe de hacer el programa es hacer la traslación de una figura, los puntos de la figura se deben de graficar como guste el usuario y el cual una vez graficada la figura el usuario pueda hacer la traslación desde uno de los puntos ya sea izquierda, derecha, arriba o abajo y además de que pueda elegir cuantas unidades sean, desde 1 hasta el que decida salvo que no debe de ser unidades demasiado grandes para no tener complicaciones.

La figura que se debe de rotar debe de tomar el trazo de la figura recién creada mas no de la primera creada siendo así conocida como la prima de la prima.

## Herramientas a usar:

JavaScript: Para la funcionalidad del plano cartesiano, registro de los puntos, trazo de la figura, traslación de la figura, elección del movimiento de traslación, unidades a mover, etc. En pocas palabras toda la funcionalidad del proyecto.

HTML: Para la interfaz visual del proyecto.

CSS (Opción): Para el diseño de la interfaz visual del proyecto.

Se eligieron estas herramientas debido a que es más fácil interactuar y trabajar con ellas debido a que con HTML al abrir una página web ya se tiene la interfaz visual y con lo de JavaScript para toda la funcionalidad del proyecto.

Primeramente, se instala la aplicación de VSCode en el cual una vez ya instalada ya tiene las extensiones de HTML, CSS Y JS de manera predeterminada y sencilla.

Una vez teniendo VSCode instalado se crea una carpeta, una vez creada la carpeta se crean los archivos de html, css y js, como se muestra en la siguiente imagen.

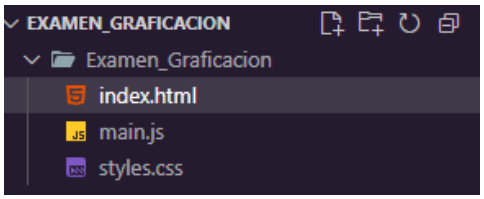



**Figura 1. Archivos html,css,js**

Elaboración propia

Una vez creado los archivos se procede a codificar, la primera parte es hacer la estructura de toda la página y en este caso es hacer el HTML.

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Plano Cartesiano Interactivo</title>
  <link rel="stylesheet" href="styles.css">
  <style>
    .scroll-wrapper {
      width: 600px;
      height: 600px;
      overflow: scroll;
      border: 1px solid #ccc;
      border-radius: 6px;
      background: #fff;
    }
  </style>
</head>
<body>
  <header>
    <h1>Graficador de Figuras y Traslaciones</h1>
  </header>
  <main>
    <section class="panel-control">
      <div class="card">
        <h3>1. Definir Puntos de la Figura</h3>
        <div class="input-group">
          <label for="x">Coordenada X:</label>
          <input type="number" id="x" step="1" placeholder="Ej: 5">
        </div>
        <div class="input-group">
          <label for="y">Coordenada Y:</label>
          <input type="number" id="y" step="1" placeholder="Ej: 3">
```

**Figura 2. Primera parte del código del html.**

Elaboración propia

En esta primera parte se define lo que son las coordenadas, además de que se manda a llamar el archivo style.css.

```html
          <input type="number" id="y" step="1" placeholder="Ej: 3">
        </div>
        <div class="actions">
          <button id="agregar" class="btn btn-primary">Agregar Punto</button>
          <button id="dibujar" class="btn btn-secondary">Crear Diagrama</button>
        </div>
        <div id="puntos" class="current-points"><strong>Puntos actuales:</strong> </div>
      </div>

      <div class="card">
        <h3>2. Aplicar Traslación</h3>
        <div class="input-group">
          <label for="figura">Figura a trasladar:</label>
          <select id="figura">
            <option value="0">Original</option>
          </select>
        </div>
        <div class="input-group">
          <label for="direccion">Dirección:</label>
          <select id="direccion">
            <option value="up">Arriba</option>
            <option value="down">Abajo</option>
            <option value="left">Izquierda</option>
            <option value="right">Derecha</option>
          </select>
        </div>
        <div class="input-group">
```

**Figura 3. Segundo bloque de código de html.**

Elaboración propia

En esta parte del bloque se muestra lo que son los botones tanto, agregar puntos, crear diagrama y también sobre la muestra de los datos.

```
        <div class="input-group">
          <label for="unidades">Unidades:</label>
          <input type="number" id="unidades" value="1" min="1">
        </div>
        <div class="actions">
          <button id="trasladar" class="btn btn-primary">Aplicar Traslación</button>
        </div>
      </div>

      <div class="card">
        <h3>3. Gestión del Proyecto</h3>
        <button id="limpiar" class="btn btn-danger">Limpiar Todo</button>
      </div>
    </section>

    <section class="canvas-container">
      <div class="scroll-wrapper">
        <canvas id="plano" width="2000" height="2000"></canvas>
      </div>
    </section>
  </main>

  <script src="main.js"></script>
</body>
</html>

```

**Figura 4. Tercer bloque del código html.**

Elaboración propia

En esta parte del código se muestra lo que es también el enlace de JavaScript.

```
const canvas = document.getElementById('plano');
const ctx = canvas.getContext('2d');
const W = canvas.width;
const H = canvas.height;
const step = 25;
const cx = W / 2;
const cy = H / 2;

let puntos = [];
let traslaciones = [];

Tabnine | Edit | Test | Explain | Document
function dibujarPlano() {
  ctx.clearRect(0, 0, W, H);

  ctx.strokeStyle = '#eee';
  ctx.lineWidth = 1;
  for (let x = 0; x <= W; x += step) {
    ctx.beginPath();
    ctx.moveTo(x, 0);
    ctx.lineTo(x, H);
    ctx.stroke();
  }
  for (let y = 0; y <= H; y += step) {
    ctx.beginPath();
    ctx.moveTo(0, y);
    ctx.lineTo(W, y);
    ctx.stroke();
  }

  ctx.strokeStyle = '#000';
  ctx.lineWidth = 2;
```

**Figura 5. Primer bloque de código de JS.**

Elaboración propia

Esta parte del código toma un elemento canvas de la página y obtiene su contexto en dos dimensiones para poder dibujar. Define el ancho y el alto del área de trabajo, así como un valor de paso que determina cada cuántos píxeles se dibujará una línea de la cuadrícula. También calcula el centro del canvas, que corresponde al origen del plano cartesiano. Se crean dos arreglos vacíos, uno pensado para guardar puntos y otro para registrar traslaciones o transformaciones que se quieran aplicar después. La función principal limpia el lienzo en cada ejecución para evitar que se acumulen trazos, dibuja una cuadrícula completa con líneas finas y grises, y luego cambia el estilo de dibujo para remarcar los ejes principales en negro y con mayor grosor. En esencia, este fragmento de código construye un plano cartesiano básico dentro del canvas y deja preparado el espacio para graficar puntos o aplicar transformaciones en el futuro.




**Figura 6. Segundo bloque de código de JS.**

Elaboración propia

Este fragmento completa la lógica del plano cartesiano. Primero dibuja los ejes principales: una línea vertical que pasa por el centro y una horizontal que lo cruza,

representando los ejes Y y X. Después configura el estilo del texto y recorre valores enteros tanto en el eje horizontal como en el vertical para colocar etiquetas numéricas en cada división de la cuadrícula, excepto en el origen. De esa forma, cada línea de la cuadrícula queda identificada con su valor correspondiente, lo que facilita leer coordenadas.

Luego aparece la definición de la función dibujarFigura. Esta función recibe un conjunto de puntos y, si hay al menos dos, prepara el contexto gráfico para dibujar una figura. Para ello guarda el estado actual del canvas, traslada el origen de coordenadas al centro del plano y aplica una escala que invierte el eje Y, de modo que el sistema de coordenadas se comporte como en matemáticas (con valores positivos hacia arriba). Finalmente, abre un nuevo trazo para comenzar a dibujar la figura con los puntos que se le pasen.

```
54  function dibujarFigura(puntos, color = '□rgba(200,0,0,0.5)', etiquetas = []) {
62    ctx.moveTo(puntos[0].x * step, puntos[0].y * step);
63    for (let i = 1; i < puntos.length; i++) {
64      ctx.lineTo(puntos[i].x * step, puntos[i].y * step);
65    }
66    ctx.closePath();
67    ctx.fillStyle = color;
68    ctx.fill();
69    ctx.strokeStyle = '□#000';
70    ctx.stroke();
71
72    ctx.restore();
73
74    ctx.font = "14px Arial";
75    ctx.fillStyle = '□#222';
76    for (let i = 0; i < puntos.length; i++) {
77      const px = cx + puntos[i].x * step;
78      const py = cy - puntos[i].y * step;
79      const label = etiquetas[i] || '';
80      ctx.fillText(`${label}(${puntos[i].x},${puntos[i].y})`, px + 5, py - 5);
81    }
82  }
83
    Tabnine | Edit | Test | Explain | Document
84  function actualizarMenuFiguras() {
85    const select = document.getElementById('figura');
86    select.innerHTML = '<option value="0">Original</option>';
87    traslaciones.forEach((fig, i) => {
88      select.innerHTML += `<option value="${i + 1}">Trasladada ${i}</option>`;
89    });
90  }
```

**Figura 7. Tercer bloque de código de JS.**

Elaboración propia

En esta parte, el programa termina de definir cómo se dibuja una figura dentro del plano. Primero toma la lista de puntos y traza líneas que los unen en orden, cerrando el camino para formar un polígono. Luego rellena esa figura con el color indicado y dibuja su contorno en negro para que quede bien delimitada. Después de restaurar el estado del contexto gráfico, se encarga de colocar etiquetas: recorre cada punto, calcula su posición real en el

canvas y escribe junto a él un texto que incluye el nombre o etiqueta asignada y sus coordenadas. De esta manera, cada vértice de la figura queda identificado visualmente.

La segunda función, llamada actualizarMenuFiguras, se ocupa de la parte interactiva de la interfaz. Busca un elemento de selección en la página y lo llena con opciones. Siempre incluye una opción llamada “Original” y, además, agrega una opción por cada figura trasladada que se haya guardado en la lista de transformaciones. Así, el usuario puede elegir desde un menú desplegable si quiere ver la figura original o alguna de sus versiones trasladadas.

```
92  function render() {
93    dibujarPlano();
94
95    const labels = [];
96    for (let i = 0; i < puntos.length; i++) {
97      labels.push(String.fromCharCode(65 + i));
98    }
99    if (puntos.length > 0) dibujarFigura(puntos, '□rgba(200,0,0,0.5)', labels);
100
101   traslaciones.forEach((obj, idx) => {
102     const sufijo = "'".repeat(idx + 1);
103     const tlabels = [];
104     for (let i = 0; i < obj.puntos.length; i++) {
105       tlabels.push(String.fromCharCode(65 + i) + sufijo);
106     }
107     dibujarFigura(obj.puntos, obj.color, tlabels);
108   });
109   actualizarMenuFiguras();
110
111   let generadosHTML = '<strong>Puntos generados:</strong> ';
112   if (traslaciones.length === 0) {
113     generadosHTML += '';
114   } else {
115     generadosHTML += traslaciones
116       .map((obj, idx) => {
117         const sufijo = "'".repeat(idx + 1);
118         return obj.puntos
119           .map((p, i) => `${String.fromCharCode(65 + i)}${sufijo} = (${p.x}, ${p.y})`)
120           .join(', ');
121       })
122       .join('<br>');
```

**Figura 8. Cuarto bloque de código de JS.**

Elaboración propia

Primero llama a la función que dibuja el plano cartesiano para limpiar y preparar el lienzo. Luego crea un arreglo de etiquetas con letras del alfabeto (A, B, C, …) que se asignan a los puntos originales. Si hay puntos definidos, dibuja la figura correspondiente en color rojo semitransparente y con esas etiquetas.

Después recorre la lista de traslaciones guardadas. A cada conjunto de puntos trasladados le asigna etiquetas derivadas de las originales, pero con apóstrofos añadidos (A', B', C'…), de modo que se distingan de la figura inicial. Cada figura trasladada se dibuja en el color que tenga asignado.

Una vez hecho esto, actualiza el menú desplegable de figuras para que el usuario pueda elegir entre la original y las trasladadas. Finalmente, construye un bloque de texto en formato HTML que muestra las coordenadas de todos los puntos generados. Si no hay traslaciones, ese bloque queda vacío; si las hay, se listan los puntos con sus etiquetas y sus coordenadas, separados por comas y con saltos de línea entre cada figura.

```
124     let divGen = document.getElementById('puntos-generados');
125     if (!divGen) {
126       divGen = document.createElement('div');
127       divGen.id = 'puntos-generados';
128       divGen.className = 'current-points';
129       document.getElementById('puntos').insertAdjacentElement('afterend', divGen);
130     }
131     divGen.innerHTML = generadosHTML;
132   }
133
      Tabnine | Edit | Test | Explain | Document
134   document.getElementById('agregar').addEventListener('click', () => {
135     const x = parseFloat(document.getElementById('x').value);
136     const y = parseFloat(document.getElementById('y').value);
137     if (!isNaN(x) && !isNaN(y)) {
138       puntos.push({ x, y });
139       document.getElementById('puntos').innerHTML =
140         '<strong>Puntos:</strong> ' + puntos.map(p => `(${p.x}, ${p.y})`).join(', ');
141     }
142   });
143
      Tabnine | Edit | Test | Explain | Document
144   document.getElementById('dibujar').addEventListener('click', () => {
145     traslaciones = [];
146     render();
147   });
148
      Tabnine | Edit | Test | Explain | Document
149   function trasladar(dx, dy, figuraIdx = 0) {
150     let base;
151     if (figuraIdx === 0) {
```

**Figura 9. Quinto bloque de código de JS.**

Elaboración propia

Primero revisa si existe un contenedor en el documento con el id puntos-generados. Si no existe, lo crea dinámicamente, le asigna un id y una clase, y lo coloca justo después del elemento con id puntos. Ese contenedor se usa para mostrar en pantalla el listado de puntos generados o trasladados. Luego, su contenido se actualiza con el HTML que se construyó previamente.

Después se añaden dos escuchadores de eventos. El primero está asociado al botón con id agregar: cuando el usuario hace clic, toma los valores numéricos de los campos de entrada x e y, los convierte a números y, si son válidos, los agrega al arreglo de puntos. Al mismo tiempo actualiza en la página el listado de puntos actuales. El segundo escuchador está en el botón con id dibujar: al pulsarlo, se vacía la lista de traslaciones y se llama a la función render, lo que provoca que se redibuje el plano con la figura original.

Finalmente aparece la definición de la función trasladar. Esta función está pensada para crear una nueva figura trasladada a partir de otra ya existente. Recibe como parámetros cuánto se debe mover en el eje X (dx), cuánto en el eje Y (dy), y un índice que indica cuál figura se va a usar como base. Si el índice es cero, significa que la figura base es la original; en caso contrario, se tomará alguna de las figuras trasladadas previamente. A partir de ahí, la función generará una nueva versión de la figura movida en el plano y la añadirá a la lista de traslaciones para que también pueda dibujarse y seleccionarse.

```
163     const color = `rgba(${r},${g},${b},0.5)`;
164     traslaciones.push({ puntos: nueva, color });
165     render();
166   }
167
      Tabnine | Edit | Test | Explain | Document
168   document.getElementById('trasladar').addEventListener('click', () => {
169     const dir = document.getElementById('direccion').value;
170     const u = parseInt(document.getElementById('unidades').value) || 1;
171     const figuraIdx = parseInt(document.getElementById('figura').value);
172
173     switch (dir) {
174       case 'up': trasladar(0, u, figuraIdx); break;
175       case 'down': trasladar(0, -u, figuraIdx); break;
176       case 'left': trasladar(-u, 0, figuraIdx); break;
177       case 'right': trasladar(u, 0, figuraIdx); break;
178     }
179   });
180
      Tabnine | Edit | Test | Explain | Document
181   document.getElementById('limpiar').addEventListener('click', () => {
182     puntos = [];
183     traslaciones = [];
184     document.getElementById('puntos').innerHTML = '<strong>Puntos:</strong>  ';
185     document.getElementById('x').value = '';
186     document.getElementById('y').value = '';
187     document.getElementById('unidades').value = '1';
188     render();
189   });
190
191   render();
```

**Figura 9. Sexto bloque de código de JS.**

Elaboración propia

Primero, dentro de la función de traslado, se valida cuál es la figura base: si no hay puntos, no hace nada; si el índice de figura es cero, toma la figura original; si no, busca una de las figuras trasladadas previamente. A partir de esa base genera una nueva figura sumando a cada coordenada los valores de desplazamiento en X y en Y. Para diferenciar visualmente

cada nueva figura, se crea un color aleatorio semitransparente. Esa nueva figura se guarda en la lista de traslaciones y se vuelve a llamar a la función de renderizado para que aparezca en pantalla.

Después se define el comportamiento del botón “trasladar”. Cuando el usuario hace clic, se leen la dirección elegida, el número de unidades a mover y la figura seleccionada. Según la dirección, se llama a la función de traslado con los valores adecuados: hacia arriba, abajo, izquierda o derecha.

También se programa el botón “limpiar”. Al pulsarlo, se vacían tanto la lista de puntos como la de traslaciones, se reinician los campos de entrada y se actualiza el área de texto que muestra los puntos. Finalmente, se vuelve a renderizar el plano para dejarlo en blanco.

La última línea, que invoca a render(), asegura que al cargar la página se dibuje de inmediato el plano cartesiano vacío, listo para que el usuario empiece a agregar puntos y trabajar con ellos.



```css
body {
    font-family: 'Segoe UI', 'Roboto', Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f7f6;
    color: #333;
    line-height: 1.6;
}

header {
    background-color: #2c3e50;
    color: #ecf0f1;
    padding: 20px 40px;
    text-align: center;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

h1 {
    margin: 0;
    font-size: 2.2em;
}

.subtitle {
    font-size: 1.1em;
    opacity: 0.9;
}

main {
    display: flex;
    justify-content: center;
    gap: 30px;
    padding: 30px;
    flex-wrap: wrap;
}

.panel-control {
    display: flex;
    flex-direction: column;
    gap: 20px;
    min-width: 300px;
    max-width: 400px;
}

.card {
    background-color: #ffffff;
    border-radius: 8px;
    padding: 25px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.08);
    transition: transform 0.2s ease-in-out;
}

.card:hover {
    transform: translateY(-5px);
}

h3 {
    color: #34495e;
    margin-top: 0;
    margin-bottom: 20px;
    border-bottom: 1px solid #eee;
    padding-bottom: 10px;
    font-size: 1.4em;
}
```

**Figura 10. bloque de código de CSS.**

Elaboración propia

Este código CSS define la apariencia completa de la página que contiene tu plano cartesiano y los controles. Primero establece un estilo general para el cuerpo con tipografías modernas, fondo gris claro, texto oscuro y un interlineado cómodo. El encabezado se diseña con un fondo azul oscuro, texto claro, centrado y con una sombra sutil para darle relieve. Los títulos principales y subtítulos tienen tamaños y colores diferenciados para jerarquizar la información.

La sección principal se organiza con un diseño flexible que coloca el panel de controles y el área del canvas uno al lado del otro, con espacio entre ellos y adaptabilidad para pantallas pequeñas. El panel de control se presenta como una columna de tarjetas, cada tarjeta con fondo blanco, bordes redondeados, sombra suave y un efecto de elevación al pasar el cursor. Dentro de estas tarjetas, los grupos de entrada muestran etiquetas claras y campos de formulario estilizados con bordes redondeados, colores suaves y un efecto de resalte azul al enfocarse.

Los botones se diferencian por colores según su función: azul para acciones principales, gris para secundarias y rojo para eliminar o limpiar. Todos tienen bordes redondeados, tipografía en negrita y un efecto de movimiento y cambio de color al pasar el cursor. La sección que muestra los puntos actuales se resalta con un fondo gris claro y una barra lateral azul, simulando un bloque informativo.

El contenedor del canvas también se presenta como una tarjeta blanca con sombra y bordes redondeados, centrando el lienzo en su interior. El propio canvas tiene un borde gris claro y esquinas suavizadas. Además, se incluyen reglas de diseño responsivo: cuando la pantalla es más estrecha, los elementos se apilan en columna, los botones se expanden al ancho completo y los tamaños de texto se ajustan.

Finalmente, se añade un efecto visual llamativo: tanto las tarjetas como los campos de entrada y el contenedor con scroll tienen un borde animado con un degradado multicolor que se mueve continuamente, generando un resplandor dinámico. Al pasar el cursor, ese resplandor se intensifica.

# Pruebas

Una vez ejecutando el index.html muestra de primera instancia la interfaz con unos elementos.




**Figura 11. Ejecución de la página web.**

Elaboración propia

Debajo de la página se muestran los demás elementos para poder hacer todos los procesos de la graficación y traslación de una figura que se quiera ingresar desde puntos.

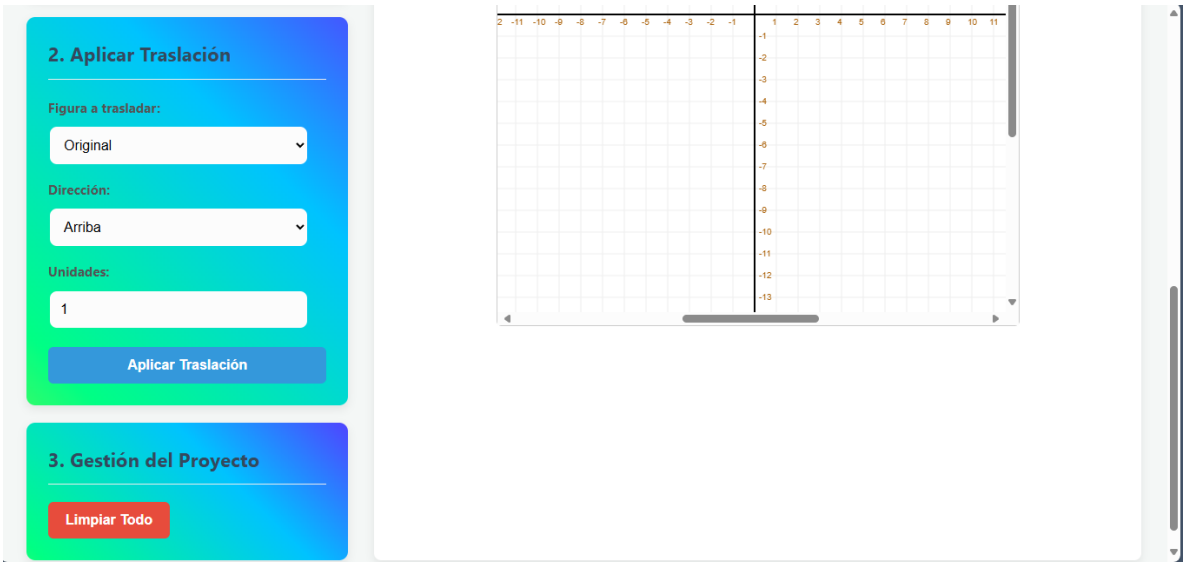



**Figura 12. Ejecución de la página web.**

Elaboración propia

Como se puede notar tiene los elementos de agregar una traslación de la figura donde queramos con un menú de dirección y cuantas unidades a requerir, ademas de un botón de eliminar todas las figuras actuales.

Para crear un diagrama se va hacia la parte de definir puntos de la figura en donde se muestra lo que son las coordenadas tanto x como y.

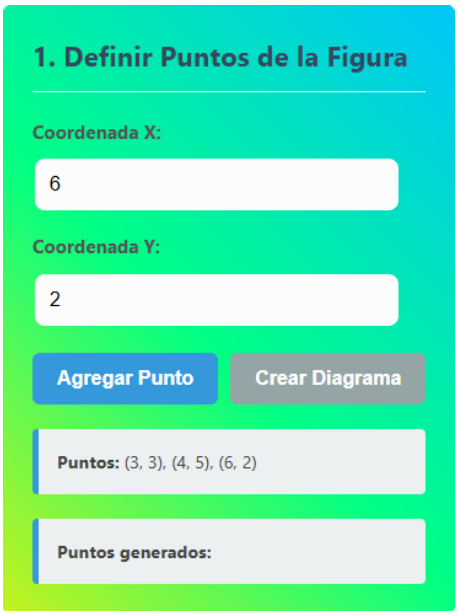



**Figura 13. Muestra de los puntos para la graficación en la página web.**

Elaboración propia

En este caso se agregaron estas tres coordenadas, aunque se pueden agregar más, como para pruebas están bien, una vez que este así se le da click en el botón que dice “crear diagrama” y si crea correctamente aparece en el plano cartesiano el diagrama como se muestra en la imagen.

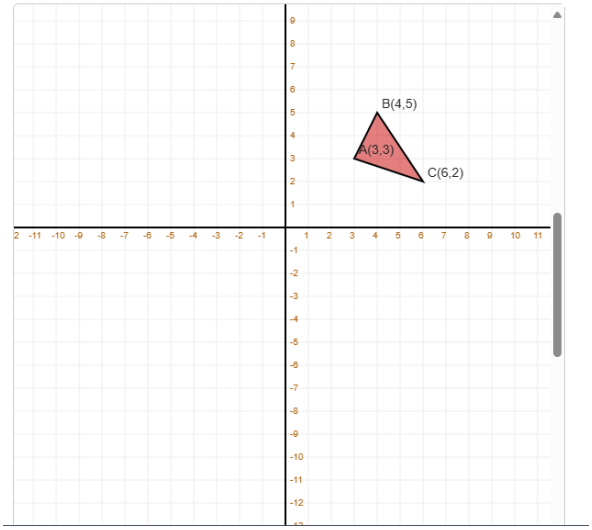



**Figura 14. Muestra de los puntos en el plano cartesiano para la graficación en la página web.**

Elaboración propia

En la figura trazada se puede observar lo que es la figura como tal y además sus coordenadas y sus puntos, para aplicar una traslación lo que se debe de hacer es ir en el apartado de traslación, como se tiene una figura pues solamente esa se va a usar, además de que se tiene lo que es la dirección y las unidades a moverse, por ejemplo, se va a mover una figura hacia la izquierda con 5 unidades, se muestra de la siguiente manera.




**Figura 15. Ejemplo de una traslación de una figura en la página web.**

Elaboración propia

Como se puede observar se agregan los valores correspondientes y si le damos en aplicar traslación una nueva figura se va a crear y estará a 5 unidades como se muestra en la siguiente imagen.

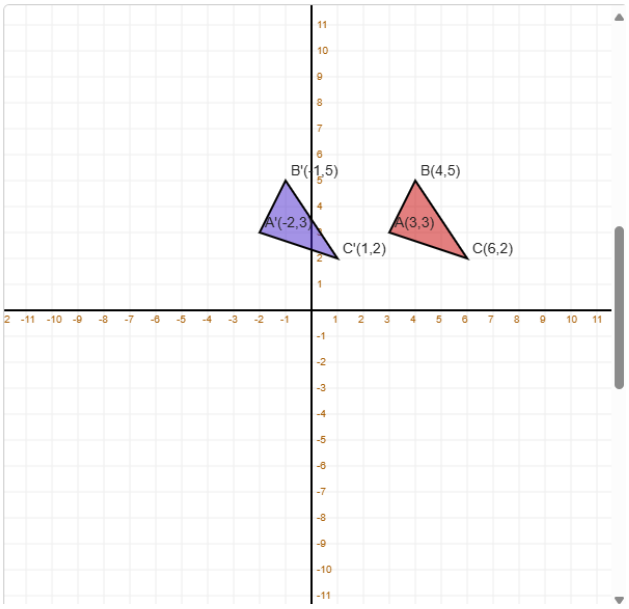



**Figura 16. Ejemplo traslación de una figura en la página web.**

Elaboración propia

En este caso como se observa se agregó una traslación con un nuevo color en la figura además de las coordenadas y también el unos apostrofes para diferenciarlo de la figura original.

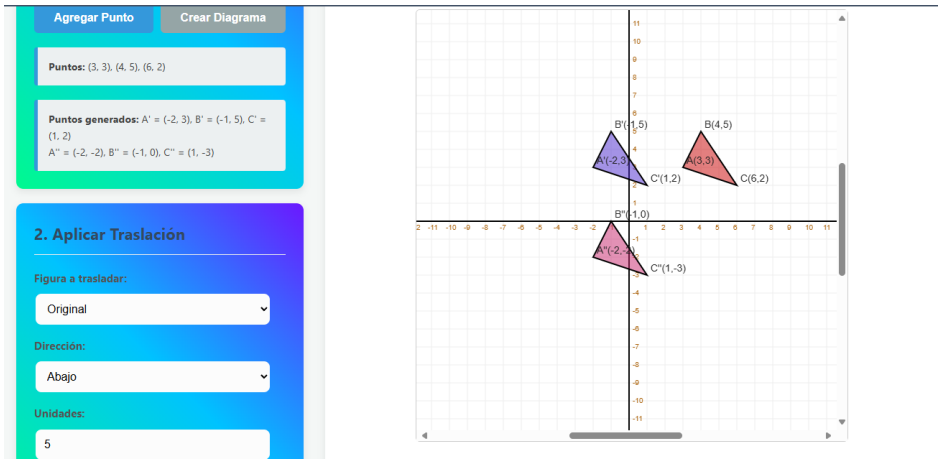



**Figura 17. Ejemplo traslación de una figura en la página web.**

Elaboración propia

En esta imagen se muestra lo que es aplicando otra traslación, pero con la figura nueva que se creo mas no tomando la original esto se hace con un menú de opciones que se tiene si se quiere crear en la figura original o en las que se han creado recientemente.

# Conclusiones

Como parte final del trabajo cabe mencionar que lo más difícil del proyecto fue la parte de la funcionalidad de JavaScript debido a que es un lenguaje diferente a las demás debido a que ocupa tener un entorno de html para funcionar correctamente, además de que para los estilos se ocupo dos tipos, uno en bloque desde el html y otro desde afuera de con un archivo llamado style.css esto con el objetivo de que la parte del scroll no sea afectada por los estilos css.